UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA

Arquitectura de hardware y software

Taller 8: diseño de la arquitectura integral para continuar la implementación

Sistema escalable de seguridad residencial

Hamed Valdez Jason Alejandro
1290897

José Isabel García Rocha

Proyecto de Carrera

7 de octubre de 2026

Repositorio: https://github.com/HamedJason/ProyectoDeCarrera

Versión del código descrita: commit 09ecf67 del repositorio del proyecto

1. Introducción y alcance

Este documento define la arquitectura integral con la que se continuará la implementación del sistema. Reúne en un solo lugar la estructura general, el despliegue, el hardware, los módulos de software, los flujos de datos, los protocolos y mensajes, la seguridad y el comportamiento ante fallas, y los relaciona con los requisitos del Taller 6 (RF-xx y RNF-xx) y con los modelos del Taller 7.

Alcance. La descripción parte del código del repositorio en el commit 09ecf67. Para no confundir lo construido con lo previsto, cada elemento lleva un estado: **Implementado** (existe en el código), **Parcial** (existe pero falta probarlo en hardware real o completar un tramo) y **Previsto** (solo está diseñado). **La cámara es un componente obligatorio del sistema.** Se describe con la placa ESP32-S3 y el sensor OV5640, que es la que tiene firmware en el repositorio. Lo que sigue abierto no es si habrá cámara sino qué módulo la implementa (sección 13); la arquitectura se diseñó para que ese cambio afecte solo al módulo de cámara.

Convención de nombres. Los diagramas usan los nombres del proyecto: nodo (ESP32-C3 con los sensores), concentrador (ESP32 que recibe por ESP-NOW y habla con el backend, llamado hub en el código), cámara (ESP32-S3), backend (Node.js), base de datos (PostgreSQL) y aplicación (la aplicación web progresiva, PWA).

2. Arquitectura general y despliegue

El sistema tiene cuatro zonas. En la **vivienda** están los nodos, el concentrador, la sirena y la cámara. Los nodos hablan con el concentrador por ESP-NOW, sin pasar por el router. Solo el concentrador y la cámara salen por Wi-Fi hacia Internet. En el **servidor virtual (VPS)** corre el backend con su base de datos dentro de Docker. El **usuario** usa la aplicación desde el teléfono y recibe los avisos por los servicios Web Push de Apple y Google.

La decisión de fondo, tomada en el Taller 4, es que **la alarma se decide en el concentrador** y no en la nube. Por eso la sirena suena aunque el servidor no responda (RNF-03), y el backend funciona como memoria, panel de control y emisor de avisos.

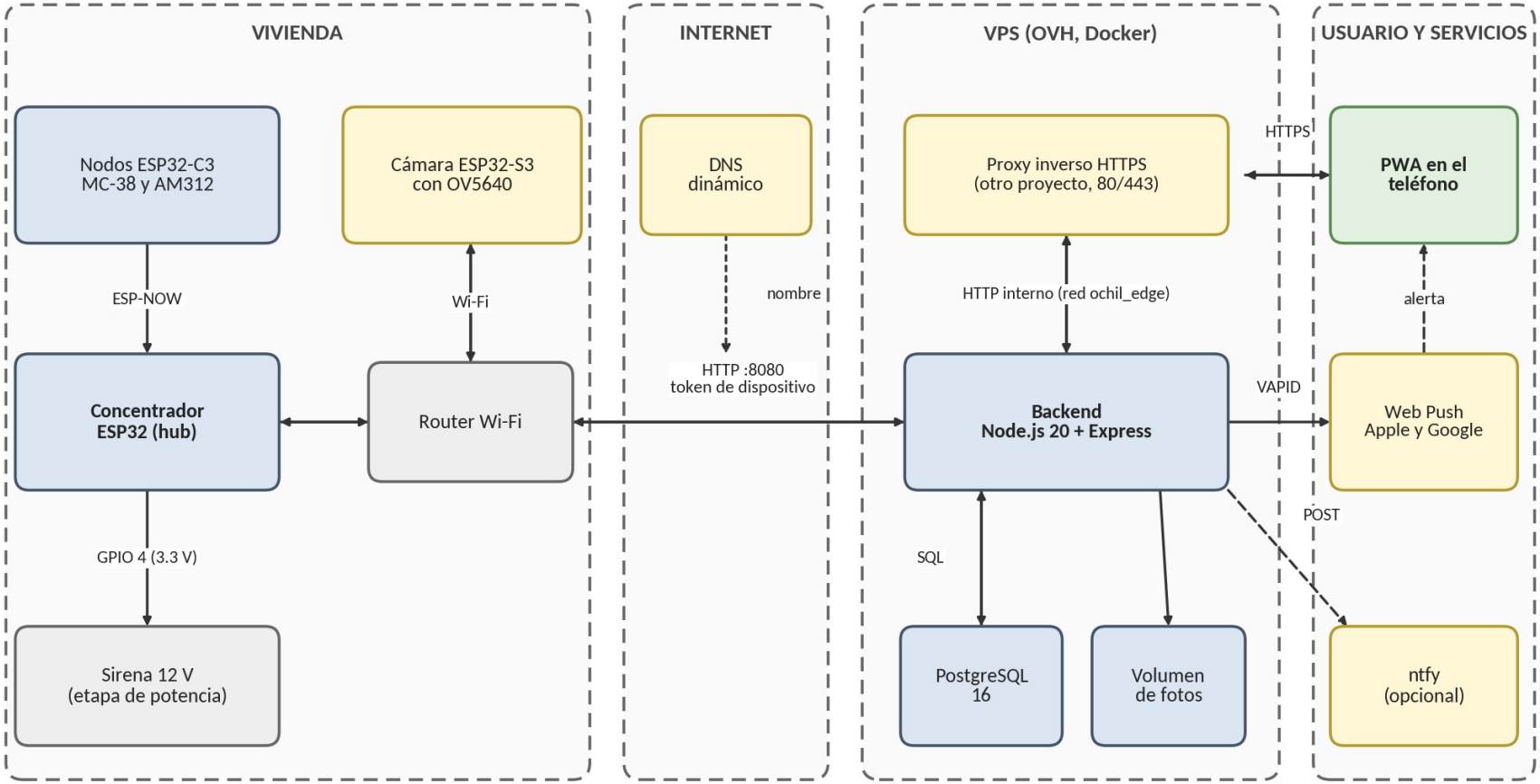


Figura 1. Arquitectura general y despliegue. En azul los componentes propios, en amarillo los servicios o equipos de terceros, en verde la aplicación del usuario y en gris el equipo pasivo.

2.1 Despliegue

El servidor ejecuta dos contenedores definidos en un solo archivo de Docker Compose. El puerto 80 y el 443 del servidor son de otro proyecto, que opera el proxy inverso con el certificado HTTPS. Por eso el backend se publica en el puerto 8080 del servidor para el concentrador y la cámara, y además se une a la red externa del otro proyecto para que su proxy lo alcance con el alias del contenedor.

Tabla 1. Elementos de despliegue.

Elemento	Tecnología	Ubicación y red	Persistencia	Estado
Nodos	ESP32-C3, firmware Arduino	Vivienda; ESP-NOW	Ninguna. Se reconstruye al encender	Implementado
Concentrador	ESP32, firmware Arduino	Vivienda; Wi-Fi 2.4 GHz	Ninguna (sin memoria no volátil)	Implementado
Cámara	ESP32-S3 con OV5640	Vivienda; Wi-Fi	Ninguna	Parcial: sin probar con una imagen aceptable
Backend	Contenedor Node.js 20 (alarma_backend)	VPS; puerto 8080 del servidor y puerto 3000 interno; red alarma y red externa ochil_edge	Archivos de fotos en el volumen datos_fotos	Implementado
Base de datos	Contenedor PostgreSQL 16 (alarma_db)	VPS; solo en la red interna alarma, sin puerto publicado	Volumen datos_postgres	Implementado
Proxy HTTPS	Servidor web del otro proyecto	VPS; puertos 80 y 443	No aplica	Implementado (de terceros)
Nombre de dominio	DNS dinámico gratuito	Internet	Exige confirmar la cuenta cada 30 días	Implementado
Aplicación	PWA servida por el backend	Teléfono del residente	Ninguna en el navegador (sin caché)	Implementado
Avisos	Web Push (VAPID) y ntfy opcional	Servicios de Apple, Google y ntfy	No aplica	Implementado

El despliegue se hace con una sola instrucción (docker compose up -d, con el archivo adicional del proxy en el servidor), lo que respalda RNF-12. La base de datos solo es visible dentro de la red interna de Docker.

3. Arquitectura de hardware

El hardware se reduce a tres placas y una sirena. Todas las placas trabajan con lógica de 3.3 V. Los pines que se muestran en la Figura 2 son los que usa el firmware actual; los de la cámara son preliminares y deben confirmarse con la placa real antes de energizar.

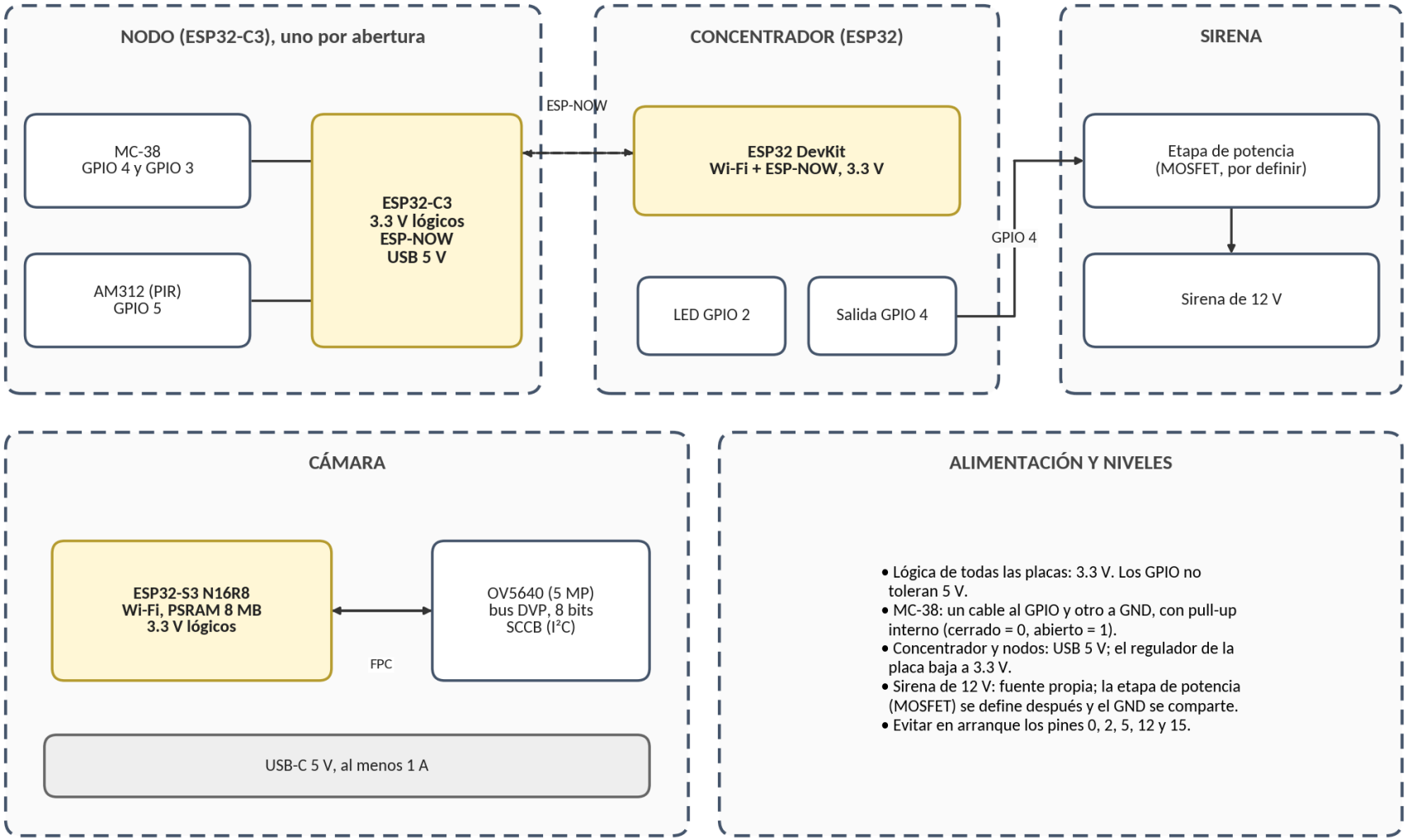


Figura 2. Diagrama de hardware con pines y niveles de voltaje.

3.1 Asignación preliminar de pines

Tabla 2. Pines de los nodos, el concentrador y la sirena.

Placa	Señal	Pin	Dir.	Nivel y modo	Observación
Nodo C3	Contacto MC-38, entrada 1	GPIO 4	Entrada	3.3 V, pull-up interno, activo bajo	Cerrado = 0, abierto = 1. Un cable al pin y otro a GND
Nodo C3	Contacto MC-38, entrada 2	GPIO 3	Entrada	3.3 V, pull-up interno, activo bajo	Opcional; el firmware admite hasta 6 contactos
Nodo C3	Salida del AM312 (PIR)	GPIO 5	Entrada	3.3 V, activo alto, sin pull-down	Con resistencia de bajada medida 1.6 V (RT-05). Tarda 30 a 60 s en estabilizarse
Nodo C3	LED de estado	sin asignar	Salida	3.3 V	Constante PIN_LED en -1; opcional
Concentrador	Salida hacia la sirena	GPIO 4	Salida	3.3 V, alto = sirena activa	Intermitente cada 300 ms. Va a la etapa de potencia de la sirena (MOSFET), cuyo diseño se define después
Concentrador	LED de estado	GPIO 2	Salida	3.3 V	LED integrado; refleja la salida de la sirena
Concentrador	Contactos locales (opcional)	4, 16, 17, 21, 22, 23	Entrada	3.3 V, pull-up, activo bajo	Desactivados (USAR_SENSORES_LOCALES = false). Si se activan no se repite el GPIO 4
Sirena	Etapa de potencia (MOSFET) y sirena de 12 V	Entrada: GPIO 4 del ESP32	Carga	12 V, fuente propia	Por definir: MOSFET de nivel lógico (compuerta a 3.3 V), resistencia de compuerta, diodo de protección y GND común

Tabla 3. Pines preliminares de la cámara ESP32-S3 con OV5640 (a verificar con la placa real).

Señal	Pin	Señal	Pin	Señal	Pin
XCLK	GPIO 15	D0 (Y2)	GPIO 11	VSYNC	GPIO 6
SIOD (SDA)	GPIO 4	D1 (Y3)	GPIO 9	HREF	GPIO 7
SIOC (SCL)	GPIO 5	D2 a D4 (Y4 a Y6)	GPIO 8, 10, 12	PCLK	GPIO 13
PWDN y RESET	sin usar (-1)	D5 a D7 (Y7 a Y9)	GPIO 18, 17, 16	Reloj XCLK	10 MHz

El firmware de la cámara trae tres mapas de pines (XIAO ESP32S3 Sense, ESP32-S3-EYE de Freenove y AI Thinker). El de la tabla corresponde al de Freenove, que es el más cercano a la placa N16R8 de la que se dispone, pero **la asignación exacta de esa placa no está verificada**. Es un punto abierto antes de seguir con la cámara.

3.2 Niveles de voltaje y alimentación

Tabla 4. Niveles de voltaje.

Punto	Voltaje	Comentario
Entrada de las placas	5 V por USB	El regulador de cada placa genera los 3.3 V. La cámara necesita una fuente de al menos 1 A

Punto	Voltaje	Comentario
Lógica de las placas	3.3 V	Los GPIO no toleran 5 V. No conectar señales de 5 V sin adaptador de nivel
MC-38 y AM312	3.3 V	El MC-38 es un contacto seco. El AM312 se alimenta y responde a 3.3 V
Sirena	12 V	Fuente separada. La etapa de potencia (MOSFET) separa la carga de 12 V del GPIO; su diseño queda pendiente
OV5640	2.8 V y 1.5 V internos	Los genera el módulo desde los 3.3 V de la placa. Un calentamiento intenso al conectar indica un defecto del módulo
Pines a evitar	0, 2, 5, 12, 15, 34 a 39	Afectan el arranque o no tienen pull-up interno. El concentrador usa el GPIO 2 solo para el LED

El nodo alimentado por batería (RNF-07) está **previsto**: el firmware actual no usa modos de bajo consumo y el consumo del nodo todavía no se ha medido.

4. Módulos de software

El software se organiza en cuatro capas que corresponden a las carpetas del repositorio: firmware, backend, base de datos y frontend. Cada módulo tiene una responsabilidad y no invade la de otro. En particular, **el firmware decide la alarma; el backend la registra y la comunica; la aplicación solo muestra y ordena**.

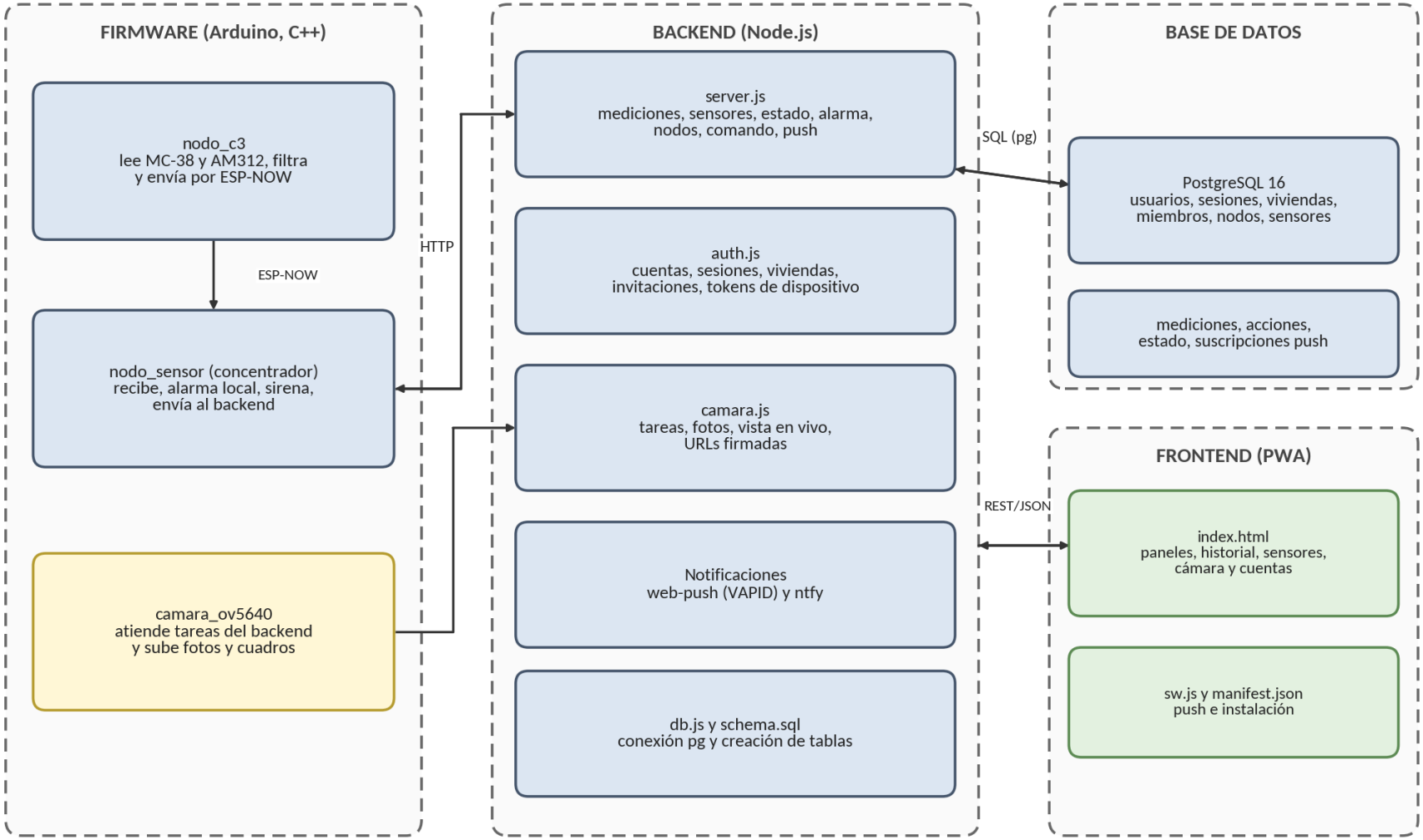


Figura 3. Diagrama de módulos de software.

Tabla 5. Responsabilidad de cada módulo.

Módulo	Responsabilidad	Estado
nodo_c3 (firmware)	Leer los contactos y el PIR. Filtrar rebotes (50 ms), bloquear el PIR 8 s entre eventos y 45 s al arrancar. Emparejarse con el concentrador (la MAC se guarda en la memoria no volátil) y buscar su canal (1 a 13). Enviar eventos y un estado cada 30 s con número de secuencia. Cifrar con AES-CCM si se activa.	Implementado
nodo_sensor (firmware del concentrador)	Guardar Wi-Fi, token y servidor por comandos seriales. Emparejar nodos y recibir sus paquetes, descartando duplicados y nodos no emparejados. Decidir la alarma con el estado armado, el modo silencioso y el tiempo. Activar la sirena 30 s. Consultar el estado al backend cada 2 s. Enviar la alarma antes que la medición, con 3 reintentos. Guardar hasta 20 eventos en RAM sin conexión. Enviar el latido de nodos cada 30 s. Reconectar el Wi-Fi sin bloquear.	Implementado
camara_ov5640 (firmware)	Iniciar la cámara con los ajustes del sensor. Consultar tareas al backend con espera larga de 20 s. Subir una foto o ráfaga, o cuadros para la vista en vivo. Reiniciarse tras 10 min sin Wi-Fi.	Parcial: sin probar con imagen aceptable
server.js	Recibir y validar mediciones, mantener el estado de armado, procesar alarmas, registrar acciones, vigilar nodos y concentrador (90 y 120 s), enviar notificaciones, servir la PWA y aplicar los encabezados de seguridad.	Implementado
auth.js	Cuentas con contraseña (scrypt), sesiones por cookie, viviendas, miembros, invitaciones, tokens de dispositivo por vivienda, límite de intentos y verificación de origen.	Implementado
camara.js	Cola de tareas por vivienda, recepción y almacenamiento de fotos, tope por días y espacio, vista en vivo con límites de tiempo y URLs firmadas de 15 min.	Implementado
Notificaciones	Enviar Web Push firmado con VAPID a las suscripciones de la vivienda y, opcionalmente, un mensaje a ntfy. Las claves VAPID se generan una vez y se guardan en la base.	Implementado
db.js y schema.sql	Conexión con pg y creación idempotente de las 16 tablas. Migra los datos de versiones anteriores.	Implementado
PostgreSQL	Almacenar usuarios, sesiones, viviendas, miembros, invitaciones, nodos, sensores, mediciones, acciones, estado, suscripciones push, cámaras, tareas, fotos y sesiones de vista en vivo.	Implementado
index.html (PWA)	Pantallas de acceso, panel de estado y control, historial, sensores y zonas, cámara, administración de viviendas e invitaciones. Consulta el estado cada 2.5 s.	Implementado
sw.js y manifest.json	Recibir y mostrar las notificaciones push y permitir la instalación en la pantalla de inicio. El service worker no guarda caché.	Implementado

5. Flujo de datos entre componentes

La Figura 4 muestra el flujo principal: un evento físico que llega hasta el teléfono. La Figura 5 muestra el flujo de control (armar y desarmar) y el de la cámara. El concentrador **siempre inicia** la comunicación con el backend (consulta cada 2 s), de modo que el router de la vivienda no necesita abrir ningún puerto.

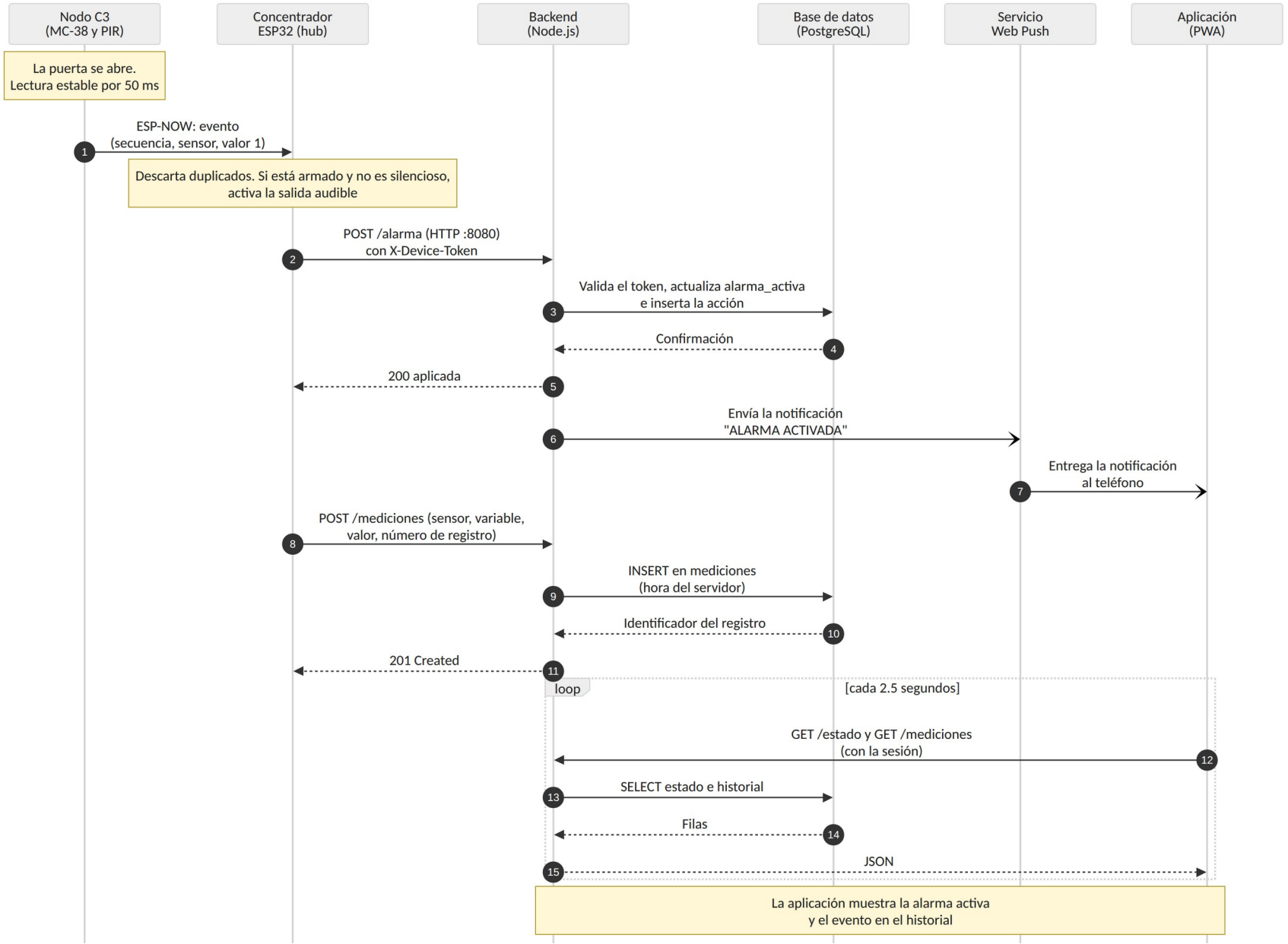


Figura 4. Flujo de una alarma: del contacto al teléfono.

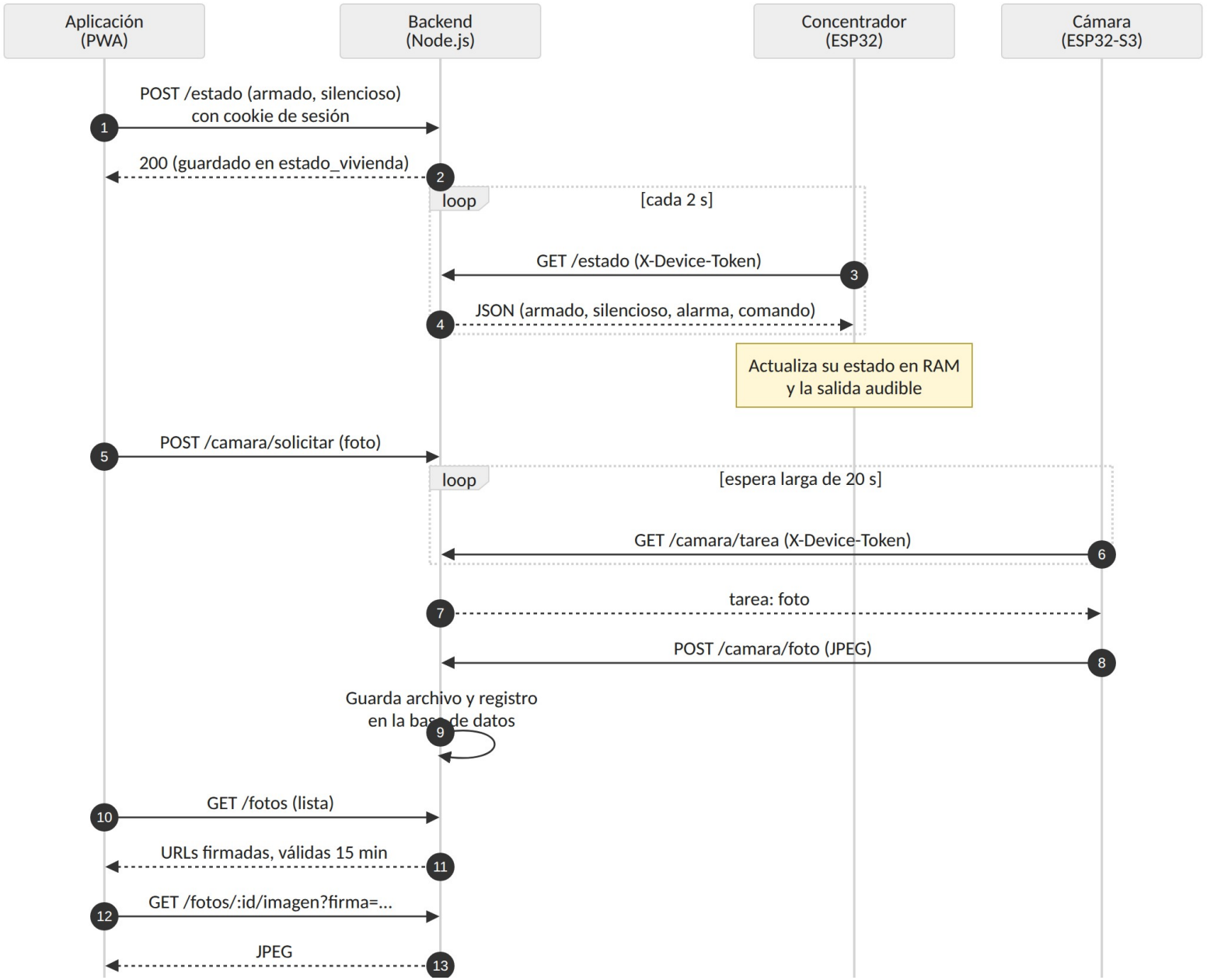


Figura 5. Flujo de control y de la cámara.

Tabla 6. Entradas y salidas de cada componente.

Componente	Entradas	Salidas
Nodo C3	Señal del MC-38 (nivel digital) y del AM312 (nivel digital). Alimentación 5 V	Paquete ESP-NOW (evento, estado, hola)
Concentrador	Paquetes ESP-NOW. Respuesta de GET /estado (armado, modo silencioso, salida manual, alarma activa)	Nivel en GPIO 4 (sirena) y GPIO 2 (LED). POST /alarma, POST /mediciones y POST /nodos/latido
Cámara	Tarea de GET /camara/tarea (foto, ráfaga o vivo). Imagen del OV5640	POST /camara/foto (JPEG) y POST /camara/cuadro (JPEG)
Backend	Peticiones HTTP de dispositivos (con token) y de la aplicación (con sesión). Variables de entorno	Respuestas JSON, imágenes y flujo multipart. Consultas SQL. Web Push y ntfy. Archivos de fotos
Base de datos	Consultas SQL del backend	Filas de las 16 tablas
Aplicación	Acciones del residente. JSON del backend. Notificaciones push	POST /estado y demás cambios. Pantallas y avisos
Sirena	Señal de 3.3 V del GPIO 4 a través de la etapa de potencia	Señal audible

6. Protocolos de comunicación

Tabla 7. Protocolos por enlace.

Enlace	Protocolo	Formato	Seguridad actual	Estado
Nodo a concentrador	ESP-NOW (IEEE 802.11, 2.4 GHz), sin conexión	Estructura binaria de 81 bytes	Lista de nodos emparejados (MAC guardadas en la memoria no volátil). AES-CCM opcional (PMK y LMK de 16 caracteres), hasta 6 pares cifrados. De fábrica desactivado	Implementado
Concentrador a backend	HTTP/1.1 sobre Wi-Fi, puerto 8080	JSON	Token de dispositivo en X-Device-Token. HTTPS opcional (USAR_HTTPS) contra el dominio con certificado, desactivado de fábrica	Implementado
Cámara a backend	HTTP/1.1, puerto 8080; espera larga de 20 s	Consulta JSON; carga binaria JPEG	Igual al concentrador	Parcial
Aplicación a backend	HTTPS por el proxy inverso (443)	REST con JSON; imagen JPEG; multipart para el vivo	Cookie HttpOnly y SameSite=Lax, verificación de origen, límite de intentos, CSP	Implementado
Proxy a backend	HTTP dentro de la red de Docker	Los mismos	Red interna que no sale del servidor	Implementado
Backend a base de datos	Protocolo de PostgreSQL (pg)	SQL parametrizado	Red interna de Docker, sin TLS (PGSSL desactivado)	Implementado
Backend a avisos	Web Push sobre HTTPS; HTTPS POST a ntfy	Mensaje cifrado de Web Push; texto	Claves VAPID. Tema de ntfy tratado como secreto	Implementado
Concentrador a backend (previsto)	MQTT sobre TLS	JSON	TLS y credencial por dispositivo	Previsto

Sobre MQTT. El diseño original prevé MQTT sobre TLS. Hoy no existe broker ni cliente MQTT en el código: el concentrador usa HTTP con consulta periódica. Se mantiene la decisión de migrar solo si se necesita enviar órdenes al concentrador sin esperar la consulta de 2 s; con la latencia actual, ese cambio no es necesario para cumplir los requisitos.

7. Formato preliminar de mensajes

7.1 Paquete ESP-NOW (nodo a concentrador)

Es una estructura empaquetada, idéntica en el nodo y en el concentrador, de 81 bytes (límite de ESP-NOW: 250). Los números se transmiten en el orden de bytes del procesador (little-endian).

Tabla 8. Campos de PaqueteSensor.

Campo	Posición	Bytes	Tipo	Descripción
magia	0	2	bytes	Marca de formato: letras "S" y "R"
version	2	1	entero sin signo	Versión del formato, actualmente 1
tipo	3	1	entero sin signo	1 evento, 2 estado periódico, 3 hola (prueba de canal), 4 emparejar (petición del nodo), 5 respuesta del concentrador
secuencia	4	4	entero sin signo	Número creciente por nodo; sirve para descartar duplicados
nodoId	8	16	texto	Identificador del nodo (se arma con su dirección MAC)
sensorId	24	24	texto	Identificador del sensor, estable
variable	48	16	texto	"estado_puerta" o "movimiento"
zona	64	16	texto	Lugar físico (por ejemplo, entrada)
valor	80	1	entero con signo	Contacto: 0 cerrado, 1 abierto. Movimiento: 1 detectado

7.2 Mensajes JSON del concentrador al backend

Todos llevan el encabezado X-Device-Token. La vivienda la determina el token, no el cuerpo. Ejemplos con valores ilustrativos:

Ejemplo 1. POST /alarma (con alarma activa).

```
{
  "vivienda_id": "casa-01", "nodo_id": "c3-a1b2c3",
  "activa": true, "motivo": "estado_puerta",
  "zona": "entrada", "sensor_id": "sensor-puerta-01",
  "silenciosa": false, "retraso_ms": 180
}
```

Ejemplo 2. POST /mediciones.

```
{
  "vivienda_id": "casa-01", "zona": "entrada",
  "nodo_id": "c3-a1b2c3", "sensor_id": "sensor-puerta-01",
  "variable": "estado_puerta", "valor": 1,
```

```
  "numero_registro": 1042
}
```

Campos obligatorios: vivienda_id, nodo_id, sensor_id, variable y valor (numérico). El backend responde 201 con el registro y la hora del servidor, o 400 con la lista de errores.

Ejemplo 3. Respuesta de GET /estado al concentrador.

```
{ "ok": true, "armado": true, "modo_silencioso": false,
  "actuador_activo": false, "alarma_activa": false }
```

Ejemplo 4. POST /nodos/latido.

```
{ "nodos": [ { "nodo_id": "c3-a1b2c3", "hace_s": 12, "rssi": -58,
    "sensores": [ { "sensor_id": "sensor-puerta-01",
      "variable": "estado_puerta", "zona": "entrada" } ] } ] }
```

Los códigos de error se manejan por el código HTTP: 400 dato inválido, 401 credencial ausente o incorrecta, 403 sin permiso, 404 no encontrado y 429 demasiados intentos (con el tiempo de espera). No se detalla más porque cada endpoint repite el mismo esquema { "ok": false, "mensaje": … }.

8. Endpoints y operaciones principales

El sistema no usa tópicos MQTT. Las operaciones son endpoints REST y tres eventos: el paquete ESP-NOW, la tarea de cámara (espera larga) y la notificación push. Se listan los principales; el resto son variantes de administración.

Tabla 9. Operaciones principales.

Operación	Quién la usa	Credencial	Función
POST /mediciones	Concentrador	Token de dispositivo	Registrar un evento o estado de sensor
POST /alarma	Concentrador	Token de dispositivo	Marcar una alarma activa o inactiva, con motivo y retraso; dispara la notificación y la foto
GET /estado	Concentrador; aplicación	Token o sesión	Leer armado, silencioso, salida manual y alarma
POST /estado	Aplicación	Sesión	Armar, desarmar, modo silencioso, salida audible o apagar alarma
POST /nodos/latido	Concentrador	Token de dispositivo	Informar nodos y sensores vistos (alta automática)
GET /mediciones y /acciones	Aplicación	Sesión	Historial de eventos y de acciones de usuarios
GET, PATCH y DELETE /sensores	Aplicación	Sesión	Ver, nombrar y reubicar sensores
GET /camara/tarea	Cámara	Token de dispositivo	Espera larga de hasta 20 s por una orden de captura
POST /camara/foto y /camara/cuadro	Cámara	Token de dispositivo	Subir una foto de alarma o un cuadro del vivo
POST /camara/solicitar, /vivo/iniciar y /vivo/detener	Aplicación	Sesión	Pedir foto o vista en vivo con tope de tiempo

Operación	Quién la usa	Credencial	Función
GET /fotos y GET /fotos/:id/imagen	Aplicación	Sesión o firma de 15 min	Listar y ver fotografías
GET /camara/vivo	Aplicación	Sesión	Flujo multipart de cuadros JPEG
POST /auth/registro, /entrar, /salir; GET /auth/yo	Aplicación	Pública o sesión	Cuenta y sesión
GET y POST /viviendas, /:id/token, /:id/invitaciones; POST /invitaciones/aceptar	Aplicación	Sesión	Viviendas, token de dispositivo, miembros e invitaciones
POST /push/suscribir, /cancelar, /prueba; GET /push/clave	Aplicación	Sesión (la clave es pública)	Suscripción a notificaciones
GET /salud	Monitoreo	Ninguna	Comprobación de disponibilidad (RNF-10)
GET y POST /comando	Firmware de la prueba de concepto	Token	Compatibilidad; usan la misma tabla de estado

9. Modelo de autenticación

Hay dos tipos de identidad y no se mezclan: las **personas** entran con cuenta y sesión; los **dispositivos** presentan un token ligado a una vivienda.

Tabla 10. Autenticación de usuarios y dispositivos.

Aspecto	Usuarios (aplicación)	Dispositivos (concentrador y cámara)
Credencial	Correo y contraseña	Token por vivienda con prefijo hv_ y 24 bytes aleatorios
Almacenamiento	Contraseña con scrypt (N = 16384, sal aleatoria de 16 bytes). Sesión: solo el hash SHA-256 del identificador	Solo el hash SHA-256 del token, con índice único por vivienda
Transporte	Cookie HttpOnly y SameSite=Lax, marcada Secure tras el proxy. Duración de 30 días con renovación	Encabezado X-Device-Token
Autorización	Pertenencia a la vivienda (propietario o miembro) en cada petición	El token fija la vivienda; lo que diga el cuerpo se ignora
Defensas	Límite de intentos (5 por cuenta en 15 min, 30 por IP); verificación de Origin en cambios; registro abierto solo con REGISTRO_ABIERTO; invitaciones con código y su propio límite	Respuesta 401 sin efecto. Comparación en tiempo constante del token global heredado
Rotación	Cambio de contraseña (POST /auth/clave)	POST /viviendas/:id/token genera uno nuevo; el anterior deja de servir
Heredado	—	El token global DEVICE_TOKEN del servidor solo se acepta para viviendas que aún no tienen token propio

Las fotos admiten además un acceso por URL firmada (HMAC SHA-256, 15 minutos) para poder mostrarlas en una notificación sin cookie. La clave de la firma se genera al arrancar el backend.

Pendiente. Los nodos no se autentican ante el backend, porque no hablan con él; se autentican ante el concentrador porque este solo acepta las MAC que emparejó (comando emparejar, ventana de 2 minutos)

y, si se activa, por cifrado ESP-NOW. El cifrado de ese enlace está desactivado y es la mejora más importante de la capa embebida.

10. Protección de credenciales y variables de entorno

La regla es que **ningún secreto vive en el repositorio**. Los secretos del servidor van en un archivo .env fuera del control de versiones y los del firmware se escriben por el puerto serie y quedan en la memoria no volátil de la placa, sin recompilar.

Tabla 11. Variables de entorno y secretos.

Variable o secreto	Dónde vive	Para qué sirve	Tratamiento
POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB	deploy/.env en el servidor	Acceso a la base de datos	No se versiona; contraseña propia, no la de ejemplo
DEVICE_TOKEN	deploy/.env	Token global heredado	No se versiona; se prefiere el token por vivienda
VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY, VAPID_SUBJECT	Variables o tabla ajustes_servidor	Firma de Web Push	Se generan una vez; la privada no sale del servidor
NTFY_TOPIC, NTFY_URL	deploy/.env	Avisos opcionales	El tema funciona como contraseña; no se publica
REGISTRO_ABIERTO, MODO_PRUEBAS	deploy/.env	Control de altas y de borrado de historial	Valores seguros por defecto (falso)
FOTOS_*, VIVO_*, CAMARA_RAFAGA, PGSSL, PORT	deploy/.env o compose	Topes de espacio y tiempo, red	No son secretos
Contraseña del Wi-Fi y token de la vivienda	Memoria no volátil (NVS) de la placa, escritos por comandos seriales	Conexión y autenticación del concentrador y la cámara	El código no contiene ningún valor; el comando ver los muestra ocultos
PMK y LMK de ESP-NOW	Constantes en el firmware (opcional)	Cifrado del enlace	Los valores del repositorio son de ejemplo y deben cambiarse antes de activar el cifrado

Medidas adicionales: el archivo .gitignore excluye .env; el contenedor del backend recibe los valores solo por variables de entorno del compose; el hash, y no el valor, es lo que se guarda de sesiones y tokens; y las respuestas de error no incluyen secretos.

Limitaciones reconocidas (RNF-09). El Wi-Fi y el token ya no están en el código fuente ni en el binario: se escriben por comandos seriales (wifi, clave, token, servidor) y viven en la memoria no volátil. Queda el riesgo de que quien tenga la placa los lea de la flash, porque aún no se activa el cifrado de flash; se mitiga con la rotación del token desde la aplicación. Debe repetirse la búsqueda de patrones sobre el historial de Git antes de la entrega.

11. Comportamiento sin Internet y después de un reinicio

La Figura 6 resume los tres escenarios. El criterio de diseño es que **ninguna falla de la red o del servidor puede apagar la protección local**, aunque sí puede retrasar los avisos.

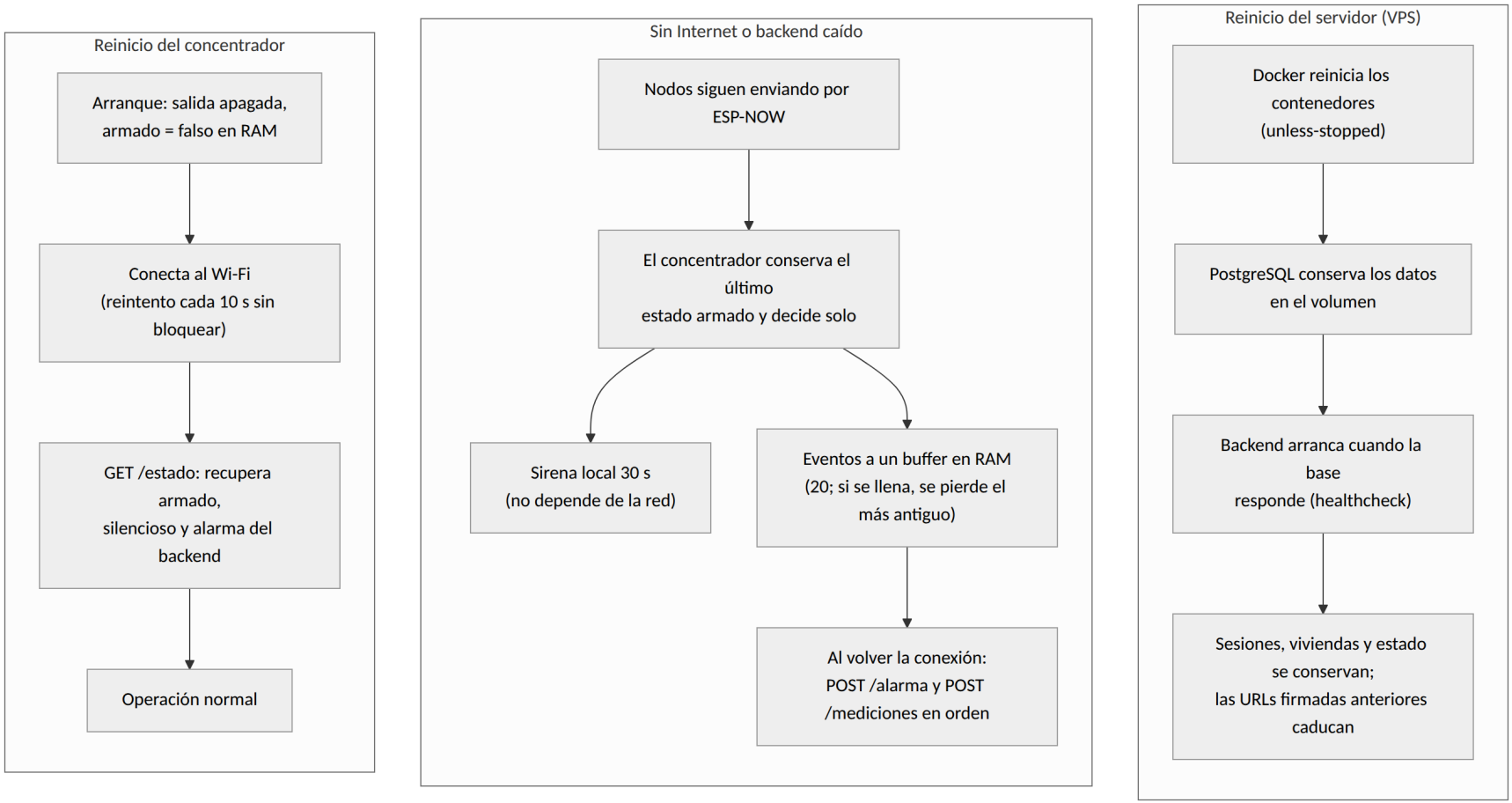


Figura 6. Sin Internet, reinicio del concentrador y reinicio del servidor.

Tabla 12. Comportamiento por componente.

Situación	Componente	Comportamiento
Sin Internet o backend caído	Nodo	Sigue enviando por ESP-NOW; no depende del router
	Concentrador	Conserva en RAM el último estado de armado y decide solo. La sirena suena 30 s. Guarda hasta 20 eventos y, si se llena, descarta el más antiguo. Reintenta el Wi-Fi cada 10 s sin bloquear. Al volver envía primero /alarma y luego /mediciones
	Backend	Marca el concentrador como desconectado si pasan 90 s sin consultas y un nodo si pasan 120 s sin latido
	Aplicación	No carga sin conexión (no hay caché). Con la página abierta, muestra el error de red. No hay aviso push si el backend no recibe la alarma
Reinicio del concentrador	Concentrador	Arranca desarmado y con la sirena apagada; no guarda nada en memoria no volátil. Hasta la primera respuesta de GET /estado no activa sirena. Pierde los eventos pendientes en RAM
	Nodo	Pierde el enlace; tras 3 envíos fallidos vuelve a buscar el canal (1 a 13; no repite el emparejamiento) y reenvía su estado cada 30 s, lo que le devuelve al concentrador el estado de los sensores
Reinicio del nodo	Nodo	Espera 45 s la estabilización del PIR y reanuda el envío; la secuencia reinicia en cero
Reinicio del servidor	Backend y base	Docker los reinicia (unless-stopped). El backend espera a que la base responda. Se conservan usuarios, viviendas, estado, historial y fotos. Las URLs firmadas anteriores dejan de servir
Reinicio de la cámara	Cámara	Se reconfigura y vuelve a consultar tareas. Se reinicia sola tras 10 min sin Wi-Fi

Brecha detectada. Si el concentrador se reinicia con la vivienda armada y sin conexión, queda desarmado hasta que recupere la red. Es la principal debilidad de esta etapa. Se corrige guardando el estado de armado en la memoria no volátil (NVS) y restaurándolo al arrancar; es un cambio pequeño y se propone como primera tarea de la siguiente etapa.

12. Tecnologías principales y versiones

Tabla 13. Tecnologías y versiones.

Capa	Tecnología	Versión	Uso
Firmware	Arduino con núcleo ESP32 (Espressif)	Núcleo ESP32 3.x y ArduinoJson 7 (confirmar al compilar)	Nodos, concentrador y cámara
Firmware	ESP-NOW y Wi-Fi del ESP32	Incluidos en el núcleo	Enlace nodo a concentrador
Firmware	esp_camera con sensor OV5640	Incluida en el núcleo	Captura de imagen
Backend	Node.js	20 (imagen alpine; el proyecto exige 18 o superior)	Servidor
Backend	Express	4.19 o superior de la serie 4	API REST
Backend	pg (cliente de PostgreSQL)	8.11 o superior de la serie 8	Acceso a datos

Capa	Tecnología	Versión	Uso
Backend	web-push	3.6 o superior de la serie 3	Notificaciones con VAPID
Base de datos	PostgreSQL	16 (imagen alpine)	Persistencia
Frontend	HTML, CSS y JavaScript sin framework; service worker y manifiesto	Estándares web	PWA instalable
Despliegue	Docker y Docker Compose	Compose v2	Contenedores del backend y la base
Infraestructura	VPS de OVH, DNS dinámico, proxy inverso del otro proyecto	—	Alojamiento, nombre y HTTPS
Avisos	Web Push (VAPID) y ntfy	Estándar	Notificaciones

Las versiones del firmware se confirman en el entorno de Arduino al compilar, y las del backend en package-lock.json. No se listan las bibliotecas secundarias.

13. Decisiones abiertas y riesgos

Estos puntos no cambian la arquitectura, pero sí condicionan lo que sigue de la implementación:

- **Cámara (obligatoria): falta definir el módulo.** El OV5640 actual se calienta mucho y la imagen sigue siendo mala; puede estar dañado. Como la cámara es obligatoria, hay que elegir entre reemplazar el módulo (otra placa ESP32-S3 con OV5640) o usar una cámara IP comercial, por ejemplo la Tapo TC70 de unos 400 pesos. La arquitectura admite ambas porque la cámara solo interactúa con tres operaciones del backend (tarea, foto y cuadro). La cámara IP usa RTSP y ONVIF, no ofrece captura HTTP y exigiría un puente en el backend con ffmpeg o un puerto abierto en el router, con riesgos de seguridad y una dependencia del operador de Internet que no está verificada.
- **Salida de la sirena.** Queda fijada en el GPIO 4 del ESP32. Falta diseñar la etapa de potencia con MOSFET (sección 3).
- **Estado de armado sin persistencia.** Ver sección 11; la corrección propuesta es la memoria no volátil del concentrador.
- **Cifrado de ESP-NOW desactivado y HTTP en el tramo del concentrador.** El token viaja sin cifrar por Internet en la configuración actual. Se activa con solo escribir una URL https:// en el comando servidor, cuando el certificado del dominio del firmware esté disponible.
- **Secretos en el firmware.** Ver sección 10.
- **Sin caché en la aplicación.** El service worker solo maneja push. Una caché del armazón de la aplicación mejoraría el comportamiento sin conexión.
- **Dependencia del proxy compartido y del dominio gratuito.** Si el otro proyecto se detiene, la aplicación deja de abrir por HTTPS; el dominio gratuito exige confirmar la cuenta cada 30 días (RE-03).
- **Retardo de entrada y salida (RF-22) y detección de personas (RF-19).** Siguen previstos y sin implementar.

14. Matriz de requisitos contra componentes

● implementa el requisito; ○ lo implementará (previsto). Desp. es el servidor, los contenedores y el repositorio.

Tabla 14. Matriz de trazabilidad entre requisitos y componentes.

Requisito	Nodo	Conc.	Cám.	Back.	BD	App	Desp.
RF-01 Detección de puertas y ventanas	●						
RF-02 Detección de movimiento	●						
RF-03 Filtro de rebotes	●						
RF-04 Identificación del evento	●	●		●	●		
RF-05 Transmisión al backend		●		●			
RF-06 Operación sin Internet		●					
RF-07 Sincronización posterior		●		●			
RF-08 Validación de mediciones				●			
RF-09 Almacenamiento persistente				●	●		
RF-10 Consulta del histórico				●	●	●	
RF-11 Armar y desarmar		●		●	●	●	
RF-12 Salida audible		●					
RF-13 Modo silencioso		●		●		●	
RF-14 Captura de fotografía			●	●			
RF-15 Alerta sin esperar la foto			●	●			
RF-16 Consulta de la fotografía				●	●	●	
RF-17 Alta de viviendas y zonas				●	●	●	
RF-18 Notificación push				●		●	
RF-19 Detección de personas (opcional)				○			
RF-20 Vista en vivo			●	●		●	
RF-21 Límite del video				●			
RF-22 Retardo de entrada y salida		○					
RF-23 Enlace confiable nodo-concentrador	●	●					
RF-24 Estado y edición de sensores				●	●	●	
RF-25 Detección de nodo perdido		●		●	●	●	
RNF-01 Detección confiable	●						
RNF-02 Falsos positivos limitados	●						
RNF-03 Alarma local sin red		●					
RNF-04 Registro remoto en 10 s		●		●			
RNF-05 Sin pérdida de eventos		●		●	●		
RNF-06 Crecimiento sin rediseño				●	●		
RNF-07 Autonomía de los nodos	○						
RNF-08 Rechazo sin credencial				●			
RNF-09 Credenciales fuera del repositorio		●	●				●
RNF-10 Disponibilidad del servicio				●			●
RNF-11 Registro de armado y desarmado				●	●		
RNF-12 Despliegue reproducible							●
RNF-13 Consumo de video acotado			●	●			
RNF-14 HTTPS y aplicación instalable						●	●